

Contents

Modules et Packages	3
1 Pourquoi modulariser un projet ?	3
2 Module : définition concrète	3
3 Les différentes formes d'import	4
Import complet	4
Import ciblé	4
Import avec alias	4
Import élément avec alias	5
4 Espace de noms (namespace)	5
5 sys.path et résolution d'import	5
6 Module exécutable vs importable	6
7 Organisation en Package	6
8 all dans un package	7
9 Import relatif	7
Cas réel structuré : mini-projet	8
11 Rechargement d'un module	8
12 Eviter les imports circulaires	8
13 Module de configuration professionnel	9
14 Tests internes simples	9
15 Structuration recommandée projet professionnel	10
16 Convention de nommage	10
17 Bonnes pratiques	10
18 Exemple avancé : Package métier	11
19 Différence module interne vs externe	12
20 Résumé opérationnel	12
Exercices de production	12
 20 Questions Intelligentes — Modules et Packages	 13
 1. Pourquoi éviter <code>from module import *</code> ?	 13
Énoncé	13
Solution	13
 2. Que se passe-t-il réellement lors d'un import ?	 13
Énoncé	13
Solution	13
 3. Pourquoi éviter la logique lourde au niveau global ?	 14
Énoncé	14
Solution	14
 4. Différence import absolu vs relatif	 15
Énoncé	15
Solution	15

5. Que contient réellement sys.modules ?	15
Énoncé	15
Solution	15
6. Pourquoi éviter les imports circulaires ?	15
Énoncé	15
Solution	16
7. Pourquoi utiliser all ?	16
Énoncé	16
Solution	16
8. Pourquoi éviter de nommer un fichier math.py ?	16
Énoncé	16
Solution	17
9. Différence entre module et package namespace	17
Énoncé	17
Solution	17
10. Pourquoi préférer un dossier src ?	17
Énoncé	17
Solution	18
11. Rechargement : piège courant	18
Énoncé	18
Solution	18
12. Différence exécution python fichier.py vs python -m module	19
Énoncé	19
Solution	19
13. Pourquoi éviter d'exécuter du code réseau à l'import ?	19
Énoncé	19
Solution	19
14. Module de configuration : pourquoi centraliser ?	20
Énoncé	20
Solution	20
15. Pourquoi séparer repository et services ?	20
Énoncé	20
Solution	20
16. Différence entre package et bibliothèque installée	21
Énoncé	21
Solution	21

17. Pourquoi documenter un module ?	21
Énoncé	21
Solution	21
18. Pourquoi limiter les imports au sommet du fichier ?	22
Énoncé	22
Solution	22
19. Namespace package (sans init.py)	22
Énoncé	22
Solution	22
20. Architecture propre d'un projet professionnel	23
Énoncé	23
Solution	23

Modules et Packages

Le support est propriété de **Bechir Bejaoui**

1 Pourquoi modulariser un projet ?

Dans un projet réel :

- Séparation des responsabilités
- Réduction du couplage
- Réutilisabilité
- Maintenabilité
- Tests unitaires simplifiés
- Déploiement structuré

Un module est une unité logique de code.

Un package est un regroupement hiérarchique de modules.

2 Module : définition concrète

Un module est simplement un fichier `.py`.

Exemple dans un scénario de gestion de stock : un module pour les opérations basiques sur les produits.

Contenu de **operations.py** :

```

"""
Module d'opérations mathématiques de base pour la gestion de stock.
Ce module expose des fonctions réutilisables.
"""

def somme(a, b):
    return a + b

def produit(a, b):
    return a * b

PI = 3.14159

Utilisation dans un script principal pour calculer des totaux de stock :

"""
Script principal utilisant le module operations pour gérer un stock.
"""

import operations

print("Somme de quantités :", operations.somme(4, 5))
print("Produit pour prix total :", operations.produit(3, 6))
print("Constante PI pour calculs :", operations.PI)

```

3 Les différentes formes d'import

Import complet

```

import math

print(math.sqrt(16))

```

Import ciblé

```

from math import sqrt

print(sqrt(25))

```

Import avec alias

```

import math as m

print(m.pi)

```

Import élément avec alias

```
from math import sqrt as racine

print(racine(36))
```

Éviter :

```
from math import *
```

Risque de pollution d'espace de noms.

4 Espace de noms (namespace)

Chaque module possède son propre espace de noms.

Exemple dans un scénario de gestion de stock : deux modules pour différents aspects du stock.

Arborescence :

```
stock_app/
  moduleA.py
  moduleB.py
  main.py
```

Contenu de **moduleA.py** :

```
x = 10  # Quantité en stock A
```

Contenu de **moduleB.py** :

```
x = 20  # Quantité en stock B
```

Script principal **main.py** :

```
import moduleA
import moduleB

print("Stock A :", moduleA.x)
print("Stock B :", moduleB.x)
```

Les variables ne se chevauchent pas.

5 sys.path et résolution d'import

Python cherche les modules dans :

- Répertoire courant
- PYTHONPATH

- Site-packages
- Chemins système

```
import sys

for chemin in sys.path:
    print(chemin)
```

6 Module exécutable vs importable

Exemple dans un scénario de calcul de stock.

Contenu de **calcul.py** :

```
"""
Module pour calculs de stock pouvant être exécuté ou importé.
"""

def carre(x):
    return x * x

if __name__ == "__main__":
    print("Test interne de stock :", carre(5))
```

Si exécuté directement :

```
python calcul.py
```

Le test s'affiche.

Si importé dans un script de gestion :

```
import calcul

print("Carré pour surface de stock :", calcul.carre(3))
```

Le bloc principal ne s'exécute pas.

7 Organisation en Package

Structure recommandée pour un package de gestion de clients dans une app e-commerce :

```
gestion/
  __init__.py
  clients.py
  produits.py
```

`commandes.py`

Contenu de `gestion/clients.py` :

```
def ajouter_client(nom):  
    return f"Client {nom} ajouté"
```

Contenu de `gestion/init.py` (démonstration approfondie) :

```
# __init__.py initialise le package et peut importer des éléments pour une exposition facile  
# Il définit le namespace du package et permet des imports relatifs.  
# Sans __init__.py (en Python 3.3+), c'est un namespace package, mais avec, il permet du code  
# Exemple : importer automatiquement des fonctions clés pour simplifier l'usage.
```

```
from .clients import ajouter_client
```

```
print("Package gestion initialisé") # Exécuté lors du premier import du package
```

Utilisation dans un script principal :

```
from gestion import ajouter_client
```

```
print(ajouter_client("Bechir"))
```

Démonstration de `init.py` : Le `print` s'exécute lors de l'import, montrant son rôle d'initialisation. Il peut aussi contenir des configurations ou des hooks pour le package entier.

8 all dans un package

Dans `init.py` :

```
__all__ = ["ajouter_client"]
```

Contrôle ce qui est exposé par :

```
from gestion import *
```

9 Import relatif

Dans un package :

```
from .clients import ajouter_client
```

Le point signifie : module courant.

Cas réel structuré : mini-projet

Structure pour une app de gestion de données e-commerce :

```
app/  
    __init__.py  
    services.py  
    repository.py  
    main.py
```

Contenu de **repository.py** :

```
def get_data():  
    return ["Produit A", "Produit B", "Produit C"]
```

Contenu de **services.py** :

```
from .repository import get_data  
  
def afficher_donnees():  
    data = get_data()  
    for d in data:  
        print("Element :", d)
```

Contenu de **main.py** :

```
from app.services import afficher_donnees  
  
if __name__ == "__main__":  
    afficher_donnees()
```

11 Rechargement d'un module

```
import importlib  
import operations
```

```
importlib.reload(operations)
```

Utile en développement interactif.

12 Eviter les imports circulaires

Problème :

```
moduleA importe moduleB  
moduleB importe moduleA
```

Solution :

- Extraire logique commune
- Import local dans fonction

Exemple dans un scénario e-commerce :

Contenu de **moduleA.py** :

```
def fonction():
    from moduleB import autre_fonction
    autre_fonction()
```

13 Module de configuration professionnel

Contenu de **config.py** :

```
"""
Module de configuration centralisée pour une app e-commerce.
"""
```

```
import os
```

```
DEBUG = os.getenv("DEBUG", "False") == "True"
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///local.db")
```

Utilisation :

```
import config

if config.DEBUG:
    print("Mode debug activé")
```

14 Tests internes simples

Dans un module :

```
def somme(a, b):
    return a + b

if __name__ == "__main__":
    assert somme(2, 3) == 5
    print("Tests validés")
```

15 Structuration recommandée projet professionnel

```
project/  
  src/  
    myapp/  
      __init__.py  
      services/  
      repository/  
      models/  
  tests/  
  requirements.txt  
  pyproject.toml
```

Explication de requirements.txt et pyproject.toml :

- **requirements.txt** liste les dépendances externes installées via `pip install -r requirements.txt`. Exemple contenu : `requests==2.28.0`.
- **pyproject.toml** est le standard moderne (PEP 518) pour la configuration du projet, incluant dépendances, build tools, et metadata. Exemple :

```
[tool.poetry]  
name = "myapp"  
version = "0.1.0"  
dependencies = { python = "^3.8", requests = "^2.28" }
```

16 Convention de nommage

Modules :

- minuscules
- pas d'espace
- pas de tiret
- descriptifs

Exemples :

```
gestion_clients.py  
auth_service.py  
db_connection.py
```

17 Bonnes pratiques

- Toujours documenter module
- Limiter code exécuté à l'import

- Eviter logique lourde au niveau global
 - Centraliser configuration
 - Séparer logique métier et accès données
-

18 Exemple avancé : Package métier

Structure pour un package e-commerce :

```
ecommerce/  
    __init__.py  
    cart.py  
    product.py  
    order.py
```

Contenu de **product.py** :

```
class Product:  
    def __init__(self, name, price):  
        self.name = name  
        self.price = price
```

Contenu de **cart.py** :

```
from .product import Product  
  
class Cart:  
    def __init__(self):  
        self.items = []  
  
    def add(self, product):  
        self.items.append(product)  
  
    def total(self):  
        return sum(p.price for p in self.items)
```

Utilisation :

```
from ecommerce.cart import Cart  
from ecommerce.product import Product  
  
c = Cart()  
c.add(Product("Clavier", 100))  
c.add(Product("Souris", 50))  
  
print("Total :", c.total())
```

19 Différence module interne vs externe

Module interne :

Fichier créé par vous.

Module externe :

Installé via pip (voir section 15 pour requirements.txt).

Exemple :

```
pip install requests
```

Puis :

```
import requests
```

```
response = requests.get("https://example.com")  
print(response.status_code)
```

20 Résumé opérationnel

- Module = fichier .py
 - Package = dossier avec **init.py**
 - Import structuré = lisibilité + maintenabilité
 - **name** == “**main**” = point d’entrée
 - sys.path = résolution
 - Organisation = clé des projets professionnels
-

Exercices de production

1. Créez un package **inventory** avec des modules pour produits et stock. Implémentez une fonction pour ajouter un produit et calculer le total. Testez avec un script principal.
 2. Ajoutez un module de configuration à un projet existant, en utilisant os.getenv pour des variables d’environnement.
 3. Refactorisez un script monofichier en un package avec séparation repository/services, et ajoutez des tests internes.
-

20 Questions Intelligentes — Modules et Packages

1. Pourquoi éviter `from module import *` ?

Énoncé

Pourquoi cette forme d'import peut provoquer des bugs difficiles à détecter ?

Solution

```
"""
Explication :
- from module import * injecte tous les noms publics dans l'espace courant.
- Risque de collision silencieuse avec des variables locales.
- Rend le code moins lisible et plus difficile à maintenir.
"""

# moduleA.py
x = 10

# moduleB.py
x = 20

# main.py
from moduleA import *
from moduleB import *

print(x)  # Quelle valeur ? 20 (moduleB écrase moduleA)
```

Conclusion : préférer `import module` ou `import ciblé`.

2. Que se passe-t-il réellement lors d'un import ?

Énoncé

Pourquoi un module n'est exécuté qu'une seule fois ?

Solution

```
"""
Explication :
- Lors du premier import, Python exécute le module.
"""
```

```
- Il est ensuite mis en cache dans sys.modules.  
- Les imports suivants réutilisent l'objet déjà chargé.  
"""
```

```
# demo.py  
print("Module chargé")
```

```
# main.py  
import demo  
import demo
```

Résultat : “Module chargé” s’affiche une seule fois.

3. Pourquoi éviter la logique lourde au niveau global ?

Énoncé

Pourquoi ne faut-il pas mettre du code métier directement dans le module sans protection ?

Solution

```
"""  
Explication :  
- Le code global s'exécute à chaque import.  
- Peut provoquer effets de bord.  
- Préférer encapsuler dans des fonctions.  
"""  
  
# mauvais.py  
print("Connexion base de données") # exécuté à chaque import
```

Bonne pratique :

```
def initialiser():  
    print("Connexion base de données")  
  
if __name__ == "__main__":  
    initialiser()
```

4. Différence import absolu vs relatif

Énoncé

Pourquoi privilégier l'import absolu dans les projets structurés ?

Solution

```
"""
Explication :
- Import absolu est plus clair et stable.
- Import relatif utile uniquement dans un package interne.
"""

# Absolu
from ecommerce.cart import Cart

# Relatif (dans package)
from .product import Product
```

5. Que contient réellement sys.modules ?

Énoncé

Comment vérifier qu'un module est en cache ?

Solution

```
"""
Explication :
- sys.modules est un dictionnaire des modules déjà chargés.
- Clé = nom du module, valeur = objet module.
"""

import sys
import math

print("math dans cache ?", "math" in sys.modules)
```

6. Pourquoi éviter les imports circulaires ?

Énoncé

Quel problème survient si deux modules s'importent mutuellement ?

Solution

```
"""
Explication :
- Python charge module A.
- A importe B.
- B importe A avant que A soit complètement initialisé.
- Provoque ImportError ou attribut manquant.
"""

# Solution : déplacer l'import dans la fonction.

def fonction():
    from moduleB import autre_fonction
    autre_fonction()
```

7. Pourquoi utiliser all ?

Énoncé

Quel est l'intérêt de contrôler l'export public d'un package ?

Solution

```
"""
Explication :
- __all__ définit explicitement ce qui est exporté.
- Protège l'API publique.
"""

# __init__.py
__all__ = ["ajouter_client"]
```

8. Pourquoi éviter de nommer un fichier math.py ?

Énoncé

Quel bug subtil cela peut provoquer ?

Solution

```
"""
Explication :
- Si vous créez math.py, il masque le module standard math.
- L'import import math chargera votre fichier.
"""

# math.py local
print("Mon module math")

# main.py
import math
```

Résultat inattendu : module standard inaccessible.

9. Différence entre module et package namespace

Énoncé

Pourquoi un package est plus qu'un simple dossier ?

Solution

```
"""
Explication :
- __init__.py transforme un dossier en espace de noms.
- Permet organisation hiérarchique.
"""

# dossier/
#   __init__.py
#   module.py

from dossier import module
```

10. Pourquoi préférer un dossier src ?

Énoncé

Pourquoi structurer un projet avec src/ améliore la robustesse ?

Solution

```
project/  
  src/  
    myapp/  
    tests/  
  
"""  
Explication :  
- Évite import accidentel depuis répertoire racine.  
- Simule installation réelle.  
- Favorise packaging propre.  
"""  
  
# Exemple d'import dans src/myapp/main.py  
from myapp.services import fonction
```

11. Rechargement : piège courant

Énoncé

Pourquoi `importlib.reload` ne met pas toujours à jour les références ?

Solution

```
"""  
Explication :  
- reload met à jour le module.  
- Mais les objets déjà importés via from module import X restent inchangés.  
"""  
  
from operations import somme  
import importlib  
import operations  
  
importlib.reload(operations)  
  
# somme ici reste ancienne référence
```

12. Différence exécution python fichier.py vs python -m module

Énoncé

Pourquoi python -m est recommandé dans un package ?

Solution

```
"""
Exemple :
python -m ecommerce.main
"""

"""
Explication :
- Maintient cohérence des imports.
- Permet exécution correcte des packages.
"""

# ecommerce/main.py
print("Exécuté via -m")
```

13. Pourquoi éviter d'exécuter du code réseau à l'import ?

Énoncé

Quel problème en production ?

Solution

```
"""
Explication :
- Import doit être rapide et sans effet de bord.
- Code réseau peut ralentir démarrage ou provoquer crash.
"""

# Mauvaise pratique :
# requests.get("http://api")
```

Bonne pratique :

```
def fetch_data():
    import requests
```

```
return requests.get("http://api")
```

14. Module de configuration : pourquoi centraliser ?

Énoncé

Quel avantage dans les grands projets ?

Solution

```
"""
Explication :
- Un seul point de configuration.
- Facilite changement environnement.
- Simplifie tests.
"""

# config.py
DEBUG = False
DATABASE_URL = "sqlite://db.sqlite"
```

15. Pourquoi séparer repository et services ?

Énoncé

Quel bénéfice architectural ?

Solution

repository -> accès données
services -> logique métier

```
"""
Explication :
- Respect séparation des responsabilités.
- Testabilité accrue.
- Réduction couplage.
"""

# repository.py
def get_data():
    return []
```

```
# services.py
from .repository import get_data

def process():
    return len(get_data())
```

16. Différence entre package et bibliothèque installée

Énoncé

Comment distinguer module local et module installé ?

Solution

```
import requests
print(requests.__file__)
```

Explication :

- Chemin affiché indique origine.
 - Site-packages = module installé.
-

17. Pourquoi documenter un module ?

Énoncé

Quel impact pour l'équipe ?

Solution

```
"""
Module de gestion des utilisateurs.

Fonctions principales :
- ajouter_utilisateur
- supprimer_utilisateur
"""
```

Explication :

- Facilite maintenance.

- Permet usage via `help()`.
 - Clarifie API publique.
-

18. Pourquoi limiter les imports au sommet du fichier ?

Énoncé

Quand faire exception ?

Solution

```
"""
Explication :
- Convention : imports en haut.
- Exception : import local pour éviter dépendance lourde ou cycle.
"""

def traitement():
    import math
    print(math.pi)
```

19. Namespace package (sans `init.py`)

Énoncé

Peut-on créer un package sans `init.py` ?

Solution

```
"""
Oui (Python 3.3+).

Explication :
- Permet namespace partagé.
- Utilisé dans grandes architectures distribuées.
"""

# dossier/ (sans __init__.py)
# module.py

import dossier.module
```

20. Architecture propre d'un projet professionnel

Énoncé

Proposer une organisation robuste pour un projet réel.

Solution

```
project/
  pyproject.toml
  src/
    myapp/
      __init__.py
      config.py
      services/
      repository/
      models/
  tests/
  README.md

"""
Explication :
- src pour isolation
- séparation métier/données
- tests indépendants
- packaging propre
"""

# Exemple import
from myapp.config import DEBUG
```